

acgrep: Aho-Corasick Multi-Pattern Search Tool

1. What I built.....	2
Key design decisions.....	2
Code map.....	2
2. The tool (Track B).....	3
How the algorithm sits inside the tool.....	3
Interface decisions.....	3
Worked example.....	4
Performance comparison.....	5
acgrep against GNU grep, side by side.....	6
Linux run by Claude, for comparison.....	7
Why acgrep falls behind at 10,000 patterns.....	7
3. What I learned.....	8
The "O(P)" build claim hides a factor of 256.....	8
Merging outputs at build time is what makes suffix matches work.....	8
Edge cases came from the surrounding tool, not the algorithm.....	8
Modern grep is more than Aho–Corasick.....	8
Reading line by line has a cost of its own.....	9
Lessons from the benchmarking.....	9
4. AI use.....	9
Tools and how much.....	9
What I used them for.....	9
Where the AI was wrong or unhelpful.....	9
What I understood versus took on trust.....	10
5. Conclusion.....	11
References.....	12

1. What I built

acgrep is a C++20 command-line tool, built on my own Aho–Corasick automaton, that searches text for many literal patterns in a single pass. I chose Track B (implementation plus a working tool). The repo is at <https://github.com/adyaran/acgrep>. The automaton is about 200 lines (acgrep/aho_corasick.{h,cpp}), the CLI is about 420 lines (acgrep/acgrep.cpp), and there are 27 unit tests (tests/test_aho_corasick.cpp).

The algorithm. Aho–Corasick merges all patterns into one trie, adds failure links (from each node to the longest proper suffix of its string that is also a trie prefix), and then scans the text once. Search time is $O(T + M)$ for text length T and M reported matches, independent of the number of patterns. M itself can grow with the pattern set, which matters for the benchmarks in section 2.

Key design decisions

Dense goto table. Each Node holds next[256], one entry per byte. During build() (a breadth-first pass), every missing transition is filled from the failure node's transition. The automaton becomes a full DFA, so search() performs one array lookup per byte and never follows a failure link at query time. The price is about 1 KB per trie node, and a build cost of $O(N \cdot 256)$, where N is the number of trie nodes and $N \leq P + 1$ for P total pattern bytes. Treating 256 as a constant makes this $O(P)$, but the constant is large in practice.

Output lists merged at build time. When a node is dequeued, the failure node's output indices are appended to its own. search() then only needs to walk the current node's list, with no separate dictionary-suffix-link walk.

Every pattern is independent. Duplicates get their own index and their own count. Matches are {patternIndex, start, end}, with 0-based byte offsets and an inclusive end.

Byte-based, not character-based. Indexing uses unsigned char, so UTF-8 and arbitrary bytes work inside the automaton. --ignore-case is ASCII-only: pattern and line are lowercased before they reach the automaton, while the original pattern text is kept for display.

Lifecycle enforced. addPattern() → build() → search(). Calling search() before build() or addPattern() after build() throws std::logic_error. An empty pattern throws std::invalid_argument, because it would match everywhere with end < start.

Simplifications relative to the textbook. Failure links are folded into the transition table (the DFA form) rather than followed during search, and there is no sparse representation for very large pattern sets.

Code map

Where	What
aho_corasick.cpp addPattern	Trie insertion; records the pattern index at the end node
aho_corasick.cpp build	BFS: failure links, output merging, goto completion
aho_corasick.cpp search	One state transition per byte, emit outputs

Where	What
acgrep.cpp <code>parseArguments</code>	Option parsing, validated up front
acgrep.cpp <code>buildAutomaton / processStream</code>	Pattern loading; per-line search and output modes
tests/test_aho_corasick.cpp	Classic example, overlaps, suffix chains, duplicates, error states

2. The tool (Track B)

acgrep answers "which of these N literal strings occur in these files, and where?" with one automaton and one pass per line. Running `grep -F` once per pattern would rescan the text N times; `grep -F -f patterns.txt` also does a single pass, and that is what I benchmark against below.

Why I built it. `grep`-style multi-pattern search has neat cybersecurity uses. Cybersecurity is my major and my biggest interest within the realm of CS, so I was particularly drawn to the idea of making something functional and (somewhat) relevant to that field. Users can build or reuse a pattern library of suspicious commands, indicator strings or error codes and scan logs for all of them in one pass.

What it is not suited for (yet). Because acgrep is line-based, it suits text signatures better than binary ones. A pattern file cannot contain a newline byte, a trailing `\r` is stripped, and no match can cross a line, so a binary malware signature containing byte `0x0A` would never be found. Scanning binaries properly would need a mode that reads raw blocks without line splitting and loads patterns as hex.

Platforms. The program was written on Windows in Visual Studio. I tested the GitHub release binaries on Windows and on Apple-silicon macOS. The Linux binary was only tested by Claude (Anthropic, 2026).

How the algorithm sits inside the tool

`parseArguments` collects every option first, so option order never changes behaviour. `buildAutomaton` then adds each `--pattern-text` and each non-blank line of each `--pattern-file` (lowercased under `--ignore-case`), keeping the original text in a parallel `patterns` vector so the pattern index lines up with what the user typed, and calls `build()` once.

`processStream` reads each input (file or `-` for stdin) line by line, strips a trailing `\r`, and calls `search()` on the line (lowercased if needed). Each line is searched independently, so a match never spans lines. Matches feed one of three output modes: plain (print the matching line once), `--show-matches` (one line per match with `[pattern @ start-end]`), or `--count` (per-file occurrence count per pattern).

Interface decisions

I chose explicit `--pattern-text` and `--pattern-file` flags rather than `grep`'s positional first-argument pattern. Since the point is to handle many patterns from several sources, I felt that the implementation should demand explicit flags here.

Exit codes are `grep-compatible`: 0 for a match, 1 for no match, 2 for an error. An unreadable input file does not stop the run; the other files are still searched and the exit code is 2.

`--count` counts occurrences, not lines, including overlapping matches and patterns nested inside other patterns. `--count` and `--show-matches` are rejected together, because their outputs cannot be mixed sensibly.

An empty `--pattern-text` is an error, but blank lines in a pattern file are skipped. The first is almost certainly a shell-quoting mistake, while the second is normal file formatting. Both pattern files and input tolerate CRLF line endings, so Windows and Unix files behave the same.

Finally, the tool ships as release binaries (Windows x64, Linux x64, macOS arm64) through a GitHub Actions workflow, plus CMake presets for building from source. I wanted to use this assignment to learn release tags.

Worked example

`access.log` and `patterns.txt` are the README's sample files. Claude ran this example in its own Linux environment as I don't have a readily available Linux image (`access.log` is now committed in the examples folder):

```
$ cat patterns.txt
PowerShell
cmd.exe
404
```

```
$ acgrep --ignore-case --pattern-file patterns.txt --show-matches access.log
access.log:1: [PowerShell @ 0-9] PowerShell returned 404
access.log:1: [404 @ 20-22] PowerShell returned 404
access.log:2: [PowerShell @ 0-9] powershell returned 404
access.log:2: [404 @ 20-22] powershell returned 404
access.log:3: [PowerShell @ 0-9] POWERSHELL returned 200
access.log:4: [cmd.exe @ 0-6] cmd.exe returned error
access.log:5: [cmd.exe @ 0-6] CMD.EXE returned error
access.log:6: [404 @ 16-18] GET /index.html 404
```

```
$ acgrep --pattern-file patterns.txt --count access.log
access.log: PowerShell: 1
access.log: cmd.exe: 1
access.log: 404: 3
```

The two commands differ on purpose: the first uses `--ignore-case` and the second does not, so their outputs are not meant to agree. Line 1 of the first reports two matches because the automaton emits every pattern that ends at each position, and the reported pattern is the one the user typed (`PowerShell`), although the automaton only saw the lowercase form. Without `--ignore-case`, `--count` shows `PowerShell: 1` because only line 1 has that exact casing.

Performance comparison

Summary. `acgrep`'s results matched `grep -F` in every run: the same matching lines on Linux, and the same line counts on Windows. This is good evidence that it is correct, though the Windows check was weaker. On Windows, `acgrep` was 1.3 to 4.2 times slower than `grep -F`. The gap narrowed up to 100 patterns and then widened, to 4.2 times at 10,000. Claude's Linux tests point to the number of matches, not the size of the automaton, as the main cause of the slowdown at 10,000 patterns; I did not repeat those tests on Windows.

Setup. Claude wrote the benchmark scripts (the Windows script and my Windows results are in the `bench/` folder of the repository; the Linux scripts are not), and I ran the Windows measurements myself: Windows 11 Home, AMD Ryzen 5 7600X (12 logical cores), Windows PowerShell 5.1, `acgrep` built with the `x64-release` CMake preset. The input was a generated 50 MB log of random words (about 447,000 lines), with pattern sets of 1 to 10,000 patterns, half of them words that occur in the log and half random strings that almost never do. Times are the best of 3 runs and include process start. I compared `acgrep` with GNU `grep -F` (from Git for Windows) and the built-in `findstr /L`.

Patterns	<code>acgrep</code>	<code>grep -F</code>	<code>findstr /L</code>	<code>acgrep ÷ grep</code>
1	0.42s	0.19s	0.07s	2.2×
10	0.44s	0.28s	0.52s	1.6×
100	0.74s	0.58s	3.29s	1.3×
1,000	2.56s	0.83s	8.70s	3.1×
10,000	3.81s	0.90s	1.02s	4.2×

Correctness. All three tools reported the same number of matching lines at every pattern count. This is consistent with `acgrep` being correct, but on Windows I compared line counts only, not the lines themselves or the match positions. Claude ran a stronger check on Linux: the matching line numbers from `acgrep` and from `grep -n -F` were identical for pattern sets of 1, 100, 1,000 and 10,000 patterns, and for 100 patterns the line text was identical too. I have not repeated that check on Windows.

Speed against `grep`. `acgrep` is slower than `grep -F` at every pattern count, by 1.3× to 4.2×. The gap is not consistent: it is 2.2× at one pattern, narrows to 1.6× and 1.3× at 10 and 100 patterns, then widens to 3.1× at 1,000 and 4.2× at 10,000.

Speed against `findstr`. `acgrep` is faster than `findstr` at 10, 100 and 1,000 patterns, and slower at 1 and 10,000.

Irregular timings. `findstr` takes 8.70s at 1,000 patterns but 1.02s at 10,000, and `grep` barely slows between 1,000 and 10,000. I did not investigate either. Because I report only the best of 3 runs, the table gives no measure of run-to-run spread, so small differences (such as 10 against 100 patterns) should not be over-read.

A wrong first result, and what caused it. My first Windows run showed `acgrep` at 3.4s for a single pattern, about 18 times slower than `grep` (7 to 20 times across pattern counts). That was not `acgrep`'s real speed. My local `out\build\x64-release` folder was configured as a Debug build (`CMakeCache.txt`

said `CMAKE_BUILD_TYPE:STRING=Debug`), even though the `x64-release` preset asks for Release, because CMake keeps the value already cached in an existing build folder. I found this by timing the same read-and-search loop as a standalone program, which took 0.40s against 3.4s for `acgrep.exe`. After deleting the folder and rebuilding, `acgrep` ran the same test in 0.42s. The GitHub release workflow builds from a fresh folder with `-DCMAKE_BUILD_TYPE=Release`, so the published binaries were not affected.

acgrep against GNU grep, side by side

The `grep` column is what the GNU `grep` 3.11 manual says (Free Software Foundation, 2023); the last column is from my benchmarks. The Windows `grep` bundled with Git for Windows is version 3.0 (2017), whose manual (Free Software Foundation, 2017) names no algorithms, so the first four rows are documented only for 3.11. `Grep`'s release notes (Free Software Foundation, 2025) suggest that the second row, Aho–Corasick for many patterns, also holds for 3.0 (section 3); the other three are unconfirmed for 3.0.

Aspect	GNU grep 3.11 (manual)	acgrep	What I measured
One fixed pattern	Boyer–Moore, which can skip text	Aho–Corasick, reads every byte	grep faster: 0.19s against 0.42s on Windows (0.06s against 0.23s on Linux)
Many fixed patterns	Aho–Corasick	Aho–Corasick	Same algorithm, yet grep faster: 0.90s against 3.81s at 10,000 patterns on Windows (0.23s against 2.50s on Linux)
Automaton design	"a set of automata designed for efficiency" (Free Software Foundation, 2023, Performance section; layout not documented)	Dense 256-entry table per node, about 1 KB each	Linux memory at 10,000 patterns: 65 MB against 8 MB
Locale	Tuned for the C locale; multibyte locales are slower	Always byte-based; ASCII-only case folding	I ran grep with <code>LC_ALL=C</code> , which is grep's best case
Reading input	Does not always read all input (for example, quiet mode stops at the first match)	Reads the whole file line by line	Claude saw grep report 0.00s when its output went to <code>/dev/null</code> ; the manual does not describe this
Beyond literal strings	Regular expressions, with a slower fallback matcher for back-references	Literal strings only	Not compared
Line handling	Original author's 2010 post (Haertel, 2010; not the manual, and older grep): searches a large buffer and looks for line boundaries only after a match	Reads and searches one line at a time	Windows diagnostic, one pattern, search included: line by line 0.40s, 1 MB blocks with manual line splitting 0.18s (line-by-line reading alone: 0.28s)

Linux run by Claude, for comparison

Claude (Anthropic, 2026) also ran the benchmark in its own Linux environment (GNU grep 3.11, 4 cores, Release build, a different random log of the same size). I did not run this myself, and the data set is not identical to my Windows one, so the numbers are not directly comparable.

Patterns	acgrep	grep -F	acgrep ÷ grep
1	0.23s	0.06s	3.8×
10	0.22s	0.18s	1.2×
100	0.30s	0.77s	0.4×
1,000	0.70s	0.64s	1.1×
10,000	2.50s	0.23s	10.9×

The two machines agree at the extremes: grep is clearly faster at one pattern and far faster at 10,000. They disagree in the middle. On Linux, acgrep was $2.5\times$ faster than grep at 100 patterns (0.30s against 0.77s), which never happened on Windows. Linux grep's own timings are also irregular: it is slower at 100 patterns than at 10,000. I have not investigated either result, and a single best-of-3 run on a different data set is not enough to tell whether it is real or noise. Because I did not run the Linux benchmark myself, I cannot vouch for it. I include it only as a secondary comparison with the caveats above, and to further show my use of AI in this assignment.

Why acgrep falls behind at 10,000 patterns

On Linux, peak memory at 10,000 patterns was about 65 MB for acgrep and 8 MB for grep, and building the automaton took about 0.09s, so there the scan, not the build, is what grows with the pattern set. I did not time the build separately on Windows. I have two theories for the slower scan. I did not test them myself, but Claude tested both on Linux, as described below.

The first is cache misses. Claude's hypothesis is that the dense 256-entry-per-node table grows large enough to fall out of the CPU cache, so most transitions become cache misses. Memory growth was measured, but cache behaviour was not profiled.

The second is the number of matches. Half of each pattern set is words that occur in the log, so as the set grows nearly every line produces many matches, and M in $O(T + M)$ grows with the pattern count. As written, `search()` reports every match on a line even in plain mode, where one match is enough to print it, whereas grep can stop at the first match on a line.

Two runs would separate these: one using only the non-occurring patterns (which keeps M near zero while the automaton still grows), and one with an early-exit path in plain mode. I did not run either, but Claude ran the first, and a related one, on Linux with the `readtest` program (search only, no printing) on the 50 MB log. One pattern took 0.23s. 10,000 patterns that never occur, with an automaton about as large as the normal set and no matches, took 0.25s. The normal 10,000-pattern set, with 6.4 million matches, took 2.01s. 4,989 patterns that all occur, with a smaller automaton and 6.3 million matches, took 2.00s. On Linux the slowdown therefore follows the number of matches, at roughly 0.3 microseconds per match, not the size of the automaton. Patterns that never match keep the scan near the root of the automaton, so

this weakens the cache theory but cannot fully rule it out. I have not repeated it on Windows, and the early-exit run is still untried.

3. What I learned

The "O(P)" build claim hides a factor of 256

Textbook Aho–Corasick builds in time linear in the total pattern length P . I first assumed my build was $O(P)$ too, and only later realised that my dense table makes it $O(N \cdot 256)$, where $N \leq P + 1$ is the number of trie nodes. That is still $O(P)$ if 256 is treated as a constant, but the constant matters here: about 1 KB per node, and about 65 MB of memory at 10,000 patterns in Claude's Linux test. The README states the bound as $O(P \times 256)$.

Merging outputs at build time is what makes suffix matches work

My first version stored only the direct terminal outputs and did not copy the failure node's outputs into each node, so a pattern like `he` inside `she` was missed. Testing showed the problem very quickly, so it was a fast fix. Copying the failure node's outputs into each node (`build`, lines 97–100) means `search` needs no suffix-link walk, and `he` inside `she` is found without extra work.

Edge cases came from the surrounding tool, not the algorithm

The empty-pattern rule, stripping `\r`, and skipping blank pattern-file lines are all in the tests or README, and none is in the textbook. The idea for many of these conditions came from Claude suggesting I read some articles about `grep`. The empty pattern cost me the most time: a PowerShell test for it never reached the check, because Windows PowerShell 5.1 silently drops empty-string arguments, and that one misunderstanding took me a full day to figure out (see section 4). It also exposed a gap in my testing: the 27 unit tests cover the automaton only, and there are no automated tests of the CLI itself. A few CTest cases that run `acgrep` on the sample files and check its output and exit code would have caught this kind of problem much sooner.

Modern `grep` is more than Aho–Corasick

I read the GNU `grep` 3.11 manual (after Claude first quoted its Performance section to me), and deliberately did not read `grep`'s source code, so I would not copy any of it by accident. The manual says `grep` uses Boyer–Moore for a single fixed pattern and Aho–Corasick for several, and that this is an implementation detail that can change (Free Software Foundation, 2023, Performance section). The 3.0 manual (Free Software Foundation, 2017), for the version I ran on Windows, names no algorithm, but `grep`'s release notes (NEWS) place the move to Aho–Corasick at version 2.26, released in 2016, before 3.0 (Free Software Foundation, 2025). So at one pattern `grep` can skip text while `acgrep` reads every byte, which explains part of that gap. At 10,000 patterns both use Aho–Corasick, so the $4\times$ to $11\times$ gap must come from elsewhere: the number of matches is the leading candidate (Claude's Linux test in section 2 followed it closely), and line-by-line reading and the memory layout are other possible contributors.

Reading line by line has a cost of its own

Mike Haertel (2010), `grep`'s original author, wrote in a mailing-list post ("why GNU `grep` is fast", on the FreeBSD-current list) that `grep` avoids breaking its input into lines: it reads raw data into a large buffer, searches the buffer, and looks for line boundaries only after a match. That post describes `grep` as it was then, and I have not checked it against 3.0 or 3.11. It matches what my Windows diagnostic found, though. Reading the file line by line and searching for one pattern took 0.40s (0.28s of that was reading alone), while reading 1 MB blocks, splitting the lines by hand and searching took 0.18s. So `acgrep`'s line-by-line design probably explains part of its gap to `grep` even at one pattern, though I did not change `acgrep` to test this.

Lessons from the benchmarking

First, always check which build type a benchmark is really measuring: a preset does not override a stale CMake cache. Claude's first explanation of the slow Windows result (that MSVC's `getline` was to blame) was wrong, and only the standalone timing test exposed the real cause. Second, GNU `grep` stops at its first match when its output goes to `/dev/null` (or `NUL`), so Claude's first Linux timings of 0.00s were wrong; the scripts now write output to a file or pipe it instead. A third, which I only saw while revising this report: a benchmark whose inputs change two things at once (here, automaton size and number of matches) cannot tell you which one caused the slowdown. Separating them, as Claude later did on Linux, showed that the number of matches was the cause there.

4. AI use

Tools and how much

Claude (Anthropic, 2026) was my main AI, used mainly for GitHub support (the repository and release workflow), coding support and test cases. I also tried ChatGPT (OpenAI, 2026) and compared its output with Claude's. Claude drafted the first version of this report, from the repository and from what I told it about the project, and helped revise it. Claude also wrote the benchmark scripts, ran the Linux benchmark and wrote the diagnostic program described below. I wrote the Aho–Corasick logic myself, after studying the algorithm on my own before implementing anything. I also wrote much of the current main method.

What I used them for

GitHub help, writing and fixing code, and generating unit tests. I was comfortable with the C++ but very unfamiliar with GitHub. Getting familiar with it was part of why I put the project on GitHub, and it is why I set up the release workflow, even though I expect the lecturer will simply build it from source.

Where the AI was wrong or unhelpful

The first problem was deprecated GitHub Actions versions. I had Claude write `release.yml`, but it used outdated versions of the actions, so I bumped them by hand afterwards (`actions/checkout` to v7 and `softprops/action-gh-release` to v3, per the commit history).

The second was clumsy ChatGPT code. When I compared Claude and ChatGPT, much of ChatGPT's logic sat in the main method. I refactored that mostly by hand into the separate parsing, automaton-building and output functions the CLI has now. Unfortunately I hadn't yet set up the repository, so I have no saved copy of ChatGPT's original code to show.

The third was a wrong test from Claude. Claude gave me a PowerShell test for the empty-pattern check, running `& $exe --pattern-text "" $log` and expecting `Error: empty patterns are not supported..` The test was wrong, not `acgrep`. Windows PowerShell 5.1 silently drops empty-string arguments when launching a native program, so `acgrep` received only `--pattern-text examples\access.log`. It took the log path as the pattern, found no input files, and correctly printed `Error: at least one input file is required.` with exit code 2. The test could never have triggered the check, because Claude assumed the empty string would arrive. The `if (pattern.empty())` guard in the CLI should handle a real empty argument, but I have not run that path on Windows; the automaton class also rejects empty patterns, with a unit test. This tiny misunderstanding took a full day to figure out.

The fourth was a wrong performance diagnosis. My first Windows benchmark showed `acgrep` taking 3.4s for a single pattern. Claude guessed that MSVC's `std::getline` on an `ifstream` was the bottleneck and presented this as its best explanation, but it was a guess and it was wrong. Claude's diagnostic program (`bench/readtest.cpp`) timed the same read-and-search loop on its own at 0.40s, which showed the code was fine. I eventually realised that my local build folder had been configured as a Debug build, and CMake's cached value overrode the preset (section 2). Claude could not have found this error, since it had no access to my Windows machine.

The fifth was a set of partly wrong claims about how `grep` works. When I asked why `grep` was faster, Claude told me from memory that GNU `grep` uses a Commentz–Walter-style search for many patterns, special routines for a single pattern, and block rather than line reading. When Claude then read the GNU `grep` 3.11 manual (Free Software Foundation, 2023), it found Boyer–Moore and Aho–Corasick instead, with no mention of Commentz–Walter or block reading, and removed the claims. My later reading showed Claude was half right: the block-reading claim matches the original author's 2010 post (Haertel, 2010), and Commentz–Walter describes `grep` before version 2.26 (Free Software Foundation, 2025), not the versions I ran (section 3). This minor hallucination was thankfully noticed quickly.

The last was a loose reading of my own benchmark numbers. A review of an earlier draft of this report found that several sentences did not match the tables. The draft said `acgrep` was close to `grep` up to 100 patterns (it is $2.2\times$ slower at one pattern), that the gap widened steadily (it narrows first), and that `acgrep` beat `findstr` only at 100 and 1,000 patterns (it also does at 10). It also described the Linux results as matching Windows while hiding that `acgrep` beat `grep` at 100 patterns on Linux. After this, I checked every figure and claim in the report against the tables and sources myself.

What I understood versus took on trust

I understand the C++ and the Aho–Corasick algorithm well, because I researched the algorithm before implementing it and wrote the automaton myself. I understand the GitHub and release-workflow side much less. I got the release workflow working with heavy AI help, and I am still learning what the pieces

do. The Linux benchmark numbers rest on Claude's runs, which I did not repeat, and my conclusion that `grep 3.0` uses Aho–Corasick for many patterns rests on the release notes, not on the 3.0 manual. While AI was used extensively for this project, I was made painfully aware through error after error that LLMs are only as effective as the human catching their inevitable and numerous mistakes.

5. Conclusion

`acgrep` meets the aim of Track B: it is a working multi-pattern search tool built on my own Aho–Corasick automaton, and its results agreed with `grep -F` on every benchmark run. It is unfortunately not competitive on speed, though this was expected. On Windows it was 1.3 to 4.2 times slower than `grep -F`, and Claude's Linux tests point to the number of matches, not the size of the automaton, as the main cost at 10,000 patterns. Potential future steps follow from that: an early-exit path in plain mode, block-based reading instead of line-by-line input, automated tests of the CLI, and a raw-byte mode for binary signatures to better align with a cybersecurity use case. This project taught me as much about measuring and checking as about the algorithm itself. The stale build cache, the broken PowerShell test and the misread tables were each found by checking a result against its source rather than trusting it. I also learned the dangers of blindly trusting AI output. The assignment goal of creating a functional program forcefully exposed me to these unfamiliar roadblocks, some of which were introduced by the very AI tools students are expected to leverage in this assignment. Overall, this project was an invaluable learning experience.

References

- Anthropic. (2026). *Claude* (Opus 5.5) [Large language model]. <https://claude.ai/>
- Free Software Foundation. (2017). *GNU Grep* (Version 3.0) [Computer software manual].
- Free Software Foundation. (2023). *GNU Grep: Print lines that match patterns* (Version 3.11) [Computer software manual]. <https://www.gnu.org/software/grep/manual/>
- Free Software Foundation. (2025). *GNU grep NEWS* [Release notes].
<https://git.savannah.gnu.org/cgit/grep.git/tree/NEWS>
- Haertel, M. (2010, August 21). Why GNU grep is fast [Mailing list post]. *freebsd-current*.
<https://lists.freebsd.org/pipermail/freebsd-current/2010-August/019310.html>
- OpenAI. (2026). *ChatGPT* [Large language model]. <https://chatgpt.com/>